

P3643R1

`std::to_signed` and `std::to_unsigned`

Published Proposal, 2025-08-13

This version:

<https://eisenwave.github.io/cpp-proposals/to-signed-unsigned.html>

Author:

[Jan Schultke<janschultke@gmail.com>](mailto:janschultke@gmail.com)

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Source:

[eisenwave/cpp-proposals](https://github.com/eisenwave/cpp-proposals)

Abstract

Add `std::to_signed` and `std::to_unsigned` function templates in the style of `std::to_underlying`.

Table of Contents

- 1 Revision history**
 - 1.1 Changes since R0
- 2 Introduction**
- 3 Impact on the standard**
- 4 Possible implementation**

5 Design decisions

6 Proposed wording

References

Normative References

Informative References

§ 1. Revision history

§ 1.1. Changes since R0

- tightened constraints in § 6 Proposed wording to exclude cv-qualified types in general, not just cv `bool`
- added `freestanding` comment to feature-test macro

§ 2. Introduction

In integer numerics and bit-manipulation code, it is common to implement functionality in terms of the corresponding signed/unsigned type. The most concise form is a function-style cast with a very short type name.

```
template<class T>
T arithmetic_shift_right(T x, int s) {
    return T(std::make_signed_t<T>(x) >> s);
}

template<class T>
T wrapping_add(T x, T y) {
    constexpr unsigned to_int_promotion_defense = 0;
    return T(to_int_promotion_defense
            + std::make_unsigned_t<T>(x)
            + std::make_unsigned_t<T>(y));
}
```

However, this is problematic for two reasons:

1. The use of C-style/function-style casts may conflict with the project's style. When `static_cast` is used instead, this code becomes substantially more verbose.
2. Repeating the type `T` violates the DRY (Don't Repeat Yourself) principle in software design. Nothing guarantees us that `x` is of type `T` when writing an expression `std::make_signed_t<T>(x)`. In larger code samples, mismatching types and variables is a bug waiting to happen. To be safe, we would have to write `std::make_signed_t<decltype(x)>(x)`. However, now we are repeating the expression `x`, so we haven't fully solved the problem.

The greater the distance between `T` and the use of `std::make_{un}signed` are, the easier it is to make a mistake.

To solve these issues, this proposal adds the function templates `std::to_signed(x)` and `std::to_unsigned(x)`, which deduce `T` from `x`. This is concise and always uses the correct type.

A GitHub code search for

```
/[a-zA-Z_](to_signed|to_unsigned)|static_cast<(typename)? ?((::)?std::)?  
(make_signed|make_unsigned)/ -is:fork language:c++
```

... shows that roughly 12.9K C++ files already use a non-standard `to_unsigned` and `to_signed`, or `static_cast` to `make_signed` or `make_unsigned`.

By comparison,

```
/[a-zA-Z_](to_underlying)|static_cast<(typename)? ?((::)?std::)?  
(underlying_type)/ -is:fork language:c++
```

... yields 30.8K C++ files which use `to_underlying` or `static_cast` to `std::underlying_type`, of which 11.6K C++ files convert via `static_cast`.

The proposal [P1682R3] for `std::to_underlying` had similar rationale, and at the time, the author was only able to discover 1000 search results for `to_underlying`.

§ 3. Impact on the standard

This proposal is a pure library extension.

NOTE: [\[ranges.syn\]](#) already defines an exposition-only function `to_unsigned_like`, however, this is more powerful than the proposed function because it operates on `unsigned_like` types, not unsigned integer types. Therefore, the wording in `[ranges]` remains unaffected.

§ 4. Possible implementation

```
template<class T>
constexpr std::make_signed_t<T> to_signed(T x) noexcept {
    return static_cast<std::make_signed_t<T>>(x);
}

template<class T>
constexpr std::make_unsigned_t<T> to_unsigned(T x) noexcept {
    return static_cast<std::make_unsigned_t<T>>(x);
}
```

§ 5. Design decisions

This proposal follows precedent: Similar to `to_underlying`, the proposed functions are located in `<utility>`. The naming scheme is based on `to_underlying` and the search result in [§ 2 Introduction](#).

§ 6. Proposed wording

The proposed wording is relative to [\[N5008\]](#).

In subclause [\[version.syn\]](#), add the following feature-testing macro:

```
#define __cpp_lib_to_signed 20XXXXL // freestanding, also in <utility>
```

In subclause [\[utility.syn\]](#), update the synopsis as follows:

```
namespace std {
    [...]
    // [utility.sign.conv], sign conversion
    template<class T>
    constexpr make_signed_t<T> to_signed(T value) noexcept;
    template<class T>
    constexpr make_unsigned_t<T> to_unsigned(T value) noexcept;

    // [utility.underlying], to_underlying
    template<class T>
        constexpr underlying_type_t<T> to_underlying(T value) noexcept;
    [...]
}
```

In subclause [\[utility\]](#), add a subclause immediately prior to [\[utility.underlying\]](#):

Sign conversion [utility.sign.conv]

```
template<class T>
    constexpr make_signed_t<T> to_signed(T value) noexcept;
```

Constraints: T is a cv-unqualified integral or enumeration type other than bool.

Returns: static_cast<make_signed_t<T>>(x).

```
template<class T>
    constexpr make_unsigned_t<T> to_unsigned(T value) noexcept;
```

Constraints: T is a cv-unqualified integral or enumeration type other than bool.

Returns: static_cast<make_unsigned_t<T>>(x).

NOTE: Because `make_unsigned_t` Mandates that T is an integral type, `to_unsigned` is arguably not SFINAE-friendly because say, `make_unsigned_t<int*>` is ill-formed without being instantiated. I don't attempt to fix this; I assume that the above specification "just works".

NOTE: The name of the subclause is based on [\[meta.trans.sign\]](#), sign modifications.

§ References

§ Normative References

[N5008]

Thomas Köppe. *Working Draft, Programming Languages — C++*. 15 March 2025. URL: <https://wg21.link/n5008>

§ Informative References

[P1682R3]

JeanHeyd Meneide. *std::to_**underlying*. 22 January 2021. URL: <https://wg21.link/p1682r3>